Sorting Algorithms

Bucket Sort

Core Idea

Distribute elements into several buckets based on their value range, **sort each bucket individually** (usually with Insertion Sort), then **concatenate all buckets** back together. Works best when input is **uniformly distributed** across a known range.

The Three Phases

- **Scatter** — distribute elements into the right buckets
- **Sort** — sort each bucket individually
- **Gather** — concatenate all buckets in order

Steps

1. Find the **min and max** values to know the range
2. Create `n` empty buckets (n = number of elements)
3. For each element, calculate **which bucket it belongs to** using a formula
4. **Insert** each element into its corresponding bucket
5. **Sort each bucket** individually using Insertion Sort
6. **Concatenate** all buckets from first to last into the final array

---

Visualizing It

```
[0.72, 0.13, 0.45, 0.88, 0.31, 0.56, 0.23]
7 elements → 7 buckets, each covering a range of 1/7 ≈ 0.14
```

Phase 1 — Scatter into buckets:

```
Bucket 0 [0.00 - 0.14): 0.13
Bucket 1 [0.14 - 0.28): 0.23
Bucket 2 [0.28 - 0.42): 0.31
Bucket 3 [0.42 - 0.56): 0.45
Bucket 4 [0.56 - 0.70): 0.56
Bucket 5 [0.70 - 0.84): 0.72
Bucket 6 [0.84 - 1.00): 0.88
```

Phase 2 — Sort each bucket:
Each bucket has only one element here — nothing to sort ✅

Phase 3 — Gather:

```
[0.13, 0.23, 0.31, 0.45, 0.56, 0.72, 0.88] ✅
```

Visualizing With Collisions

```
[0.72, 0.13, 0.45, 0.11, 0.31]
5 elements → 5 buckets
```

Phase 1 — Scatter:

```
Bucket 0 [0.00 - 0.20): 0.13, 0.11  ← collision
Bucket 1 [0.20 - 0.40): 0.31
Bucket 2 [0.40 - 0.60): 0.45
Bucket 3 [0.60 - 0.80): 0.72
Bucket 4 [0.80 - 1.00): empty
```

Phase 2 — Sort bucket 0:

```
Bucket 0: [0.11, 0.13] ✅
```

Phase 3 — Gather:

```
[0.11, 0.13, 0.31, 0.45, 0.72] ✅
```

Key Characteristics

- **Stable** — if the individual bucket sort used is stable (Insertion Sort is stable)
- **Not in-place** — needs extra arrays for buckets
- **Non comparison based** — distribution is based on value range, not comparisons

---

Time & Space Complexity

Case	Time	Why
Best	O(n + k)	Uniform distribution, one element per bucket
Average	O(n + k)	Reasonably spread across buckets
Worst	$O(n^2)$	All elements land in the same bucket, reduces to Insertion Sort

Space: O(n + k) — k buckets, n elements distributed across them

> `k` = number of buckets. Best case when `k ≈ n`.

The Optimization

The key to Bucket Sort's performance is the **bucket count and range**. Too few buckets → many collisions → approaches $O(n^2)$. Too many buckets → wasted memory. The sweet spot is **one bucket per element on average**. Also the distribution formula must map elements **uniformly** — if data is skewed, some buckets overflow while others stay empty.

How It Compares to Counting and Radix Sort

	Counting Sort	Radix Sort	Bucket Sort
Works best on	Small range integers	Large integers	Floating points / uniform data
Extra space	O(n + k)	O(n + k)	O(n + k)
Worst case	O(n + k)	O(n × d)	$O(n^2)$
Comparison based	No	No	No (but bucket sort uses one internally)

When to Use vs When to Avoid

Use When	Avoid When
Data is uniformly distributed	Data is heavily skewed
Floating point numbers	Integer range is small (use Counting Sort)
Range is known upfront	Memory is constrained
Need average O(n) performance	Worst case guarantee is needed

Why It's Asked in Interviews

Bucket Sort is asked to test whether you understand **distribution based thinking** and can identify the right sorting algorithm for the right data. Interviewers often pair it with questions about **floating point sorting** or ask you to sort data that falls within a known range — a clean signal that Bucket Sort is the intended answer.

Steps for Bucket Sort

```
Step 1: Create n buckets.

Step 2: Distribute elements into buckets.
    - Normalize to [0, 1)
    - then scale to bucket index
```

```
Step 3: Sort each bucket and concatenate.
```

Line 1: Normalize to [0.0, 1.0]

```
(num - min_val) / (max_val - min_val) squishes any range of n
umbers into a value between 0 and 1.
```

```
nums = [10, 20, 30, 40, 50]
min_val = 10, max_val = 50

10 → (10-10) / (50-10) = 0/40  = 0.0   # min always becomes
0.0
20 → (20-10) / (50-10) = 10/40 = 0.25
30 → (30-10) / (50-10) = 20/40 = 0.5
40 → (40-10) / (50-10) = 30/40 = 0.75
50 → (50-10) / (50-10) = 40/40 = 1.0   # max always becomes
1.0
```

No matter what the range is — `[10,50]` or `[-1000, 1000]` — everything maps proportionally into `[0.0, 1.0]`.

Line 2: Scale to bucket index [0, n-1]

```
int(normalized * (n-1)) stretches [0.0, 1.0] across your avai
lable buckets.
```

```
n = 5 buckets → valid indices are 0,1,2,3,4  (n-1 = 4)

0.0  * 4 = 0.0  → bucket 0
0.25 * 4 = 1.0  → bucket 1
0.5  * 4 = 2.0  → bucket 2
```

```
0.75 * 4 = 3.0  → bucket 3
1.0  * 4 = 4.0  → bucket 4
```

Why `n-1` and not `n`?

```
normalized * n  →  1.0 * 5 = 5  → bucket[5] ❌ index out of b
ounds!
normalized * (n-1) → 1.0 * 4 = 4 → bucket[4] ✅ valid last in
dex
```

Full trace

```
nums = [10, 20, 30, 40, 50],  5 buckets

10 → 0.0  → bucket[0] → [10]
20 → 0.25 → bucket[1] → [20]
30 → 0.5  → bucket[2] → [30]
40 → 0.75 → bucket[3] → [40]
50 → 1.0  → bucket[4] → [50]

concatenate → [10, 20, 30, 40, 50] ✅
```

```
nums = [10, 15, 12, 40, 50],  5 buckets

10 → 0.0  → bucket[0] → [10]
15 → 0.125 → bucket[0] → [10, 15]  # same bucket, close value
s
12 → 0.05 → bucket[0] → [10, 15, 12]
40 → 0.75 → bucket[3] → [40]
50 → 1.0  → bucket[4] → [50]

sort each bucket:
bucket[0] → [10, 12, 15]
```

```
concatenate → [10, 12, 15, 40, 50] ✅
```

In short: normalization is just proportionally mapping your actual range → `[0,1]` → bucket indices, so elements that are close in value land in the same or neighboring buckets.